API Secured, the Modern Way

OAuth2, OIDC, Rate limiting and more

Feedback

ConFoo

Hello!

Mathieu Santostefano

Tech Expert **SensioLabs**

Symfony Core Team Member

Clever Cloud Ambassador

@welcomattic.com

@welcomattic

ConFoo

Agenda to protect your API

Agenda to protect your API

Authorization with OAuth2

Agenda to protect your API

Authorization with OAuth2

Authentication with OpenID Connect (OIDC)

Agenda to protect your API

🛡️ Authorization with OAuth2

🔐 Authentication with OpenID Connect (OIDC)

⏱️ Rate Limiting

Agenda to protect your API

🛡️ Authorization with OAuth2

🔐 Authentication with OpenID Connect (OIDC)

⏱️ Rate Limiting

Each topic is explored using **Symfony solution** and a **third-party solution**.

With a surprise at the end! 🎁

What Does "Secured" Mean?

What Does "Secured" Mean?

Authorization
OAuth2

What can you do?

Roles, permissions,

access control

What Does "Secured" Mean?

Authorization
OAuth2

What can you do?

Roles, permissions,

access control

Authentication
OIDC

Who are you?

user/password → tokens

client_id/secret → tokens

What Does "Secured" Mean?

Authorization
OAuth2

What can you do?

Roles, permissions,

access control

Authentication
OIDC

Who are you?

user/password → tokens

client_id/secret → tokens

Rate Limiting

How much can you do?

Protect resources

Fair usage

Authorization

What is OAuth2?

OAuth2 is an **authorization framework** that allows **users to grant permissions to applications** to access protected resources **without disclosing credentials**.

What is OAuth2?

Today, focus on "on-behalf of user" scenario,

but OAuth2 also supports "client credentials" that is not involving user.

Refers to specification RFC 6749 to learn more.

OAuth2 simple use case

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

🔐 Alice **doesn't want to share her CloudPics password** with PhotoPrint.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

🔐 Alice **doesn't want to share her CloudPics password** with PhotoPrint.

🔗 PhotoPrint **redirects Alice to CloudPics** to authorize access.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

🔐 Alice **doesn't want to share her CloudPics password** with PhotoPrint.

🔗 PhotoPrint **redirects Alice to CloudPics** to authorize access.

🔢 Alice **grants permission**, and PhotoPrint receives a one-time valid **code**.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

🔐 Alice **doesn't want to share her CloudPics password** with PhotoPrint.

🔗 PhotoPrint **redirects Alice to CloudPics** to authorize access.

🔢 Alice **grants permission**, and PhotoPrint receives a one-time valid **code**.

🔄 PhotoPrint **exchanges the code for an access token** from CloudPics.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

🔐 Alice **doesn't want to share her CloudPics password** with PhotoPrint.

🔗 PhotoPrint **redirects Alice to CloudPics** to authorize access.

🔢 Alice **grants permission**, and PhotoPrint receives a one-time valid **code**.

🔄 PhotoPrint **exchanges the code for an access token** from CloudPics.

✅ PhotoPrint uses the **access token** to access Alice's photos on CloudPics.

OAuth2 simple use case

👩 Alice, a **user** of a photo printing service named **PhotoPrint**.

📸 PhotoPrint needs to access Alice's photos stored on a cloud service called **CloudPics**.

🔐 Alice **doesn't want to share her CloudPics password** with PhotoPrint.

🔗 PhotoPrint **redirects Alice to CloudPics** to authorize access.

🔢 Alice **grants permission**, and PhotoPrint receives a one-time valid **code**.

🔄 PhotoPrint **exchanges the code for an access token** from CloudPics.

✅ PhotoPrint uses the **access token** to access Alice's photos on CloudPics.

> 🔵 As **CloudPics API developer**, must implement OAuth2 to allow Alice granting access to photos for PhotoPrint.

OAuth2 use case

In this scenario

OAuth2 use case

In this scenario

- ☁️ *CloudPics* is the **resource server** that holds Alice's pics and the **authorization server** that issues tokens.

OAuth2 use case

In this scenario

- ☁️ *CloudPics* is the **resource server** that holds Alice's pics and the **authorization server** that issues tokens.
- 🖼️ *PhotoPrint* is the **client application** that wants to access a protected resource

OAuth2 use case

In this scenario

☁️ *CloudPics* is the **resource server** that holds Alice's pics and the **authorization server** that issues tokens.

🖼️ *PhotoPrint* is the **client application** that wants to access a protected resource

👩 *Alice* is the **resource owner** of the photos.

OAuth2 implementation in Symfony

OAuth2 implementation in Symfony

📦 **Bundle**: `league/oauth2-server-bundle`

OAuth2 implementation in Symfony

📦 **Bundle**: `league/oauth2-server-bundle`

🔧 **Integration**: Official League OAuth2 server Symfony bundle

OAuth2 implementation in Symfony

📦 **Bundle**: `league/oauth2-server-bundle`

🔧 **Integration**: Official League OAuth2 server Symfony bundle

🛡️ **Result**: **Embed a modern OAuth2 authorization server** into your Symfony API

OAuth2 implementation in Symfony

📦 **Bundle**: `league/oauth2-server-bundle`

🔧 **Integration**: Official League OAuth2 server Symfony bundle

🛡️ **Result**: **Embed a modern OAuth2 authorization server** into your Symfony API

⚡ **In action**:

Your API is the **resource server**

The bundle handles token validation and scope checking for protected endpoints

Bundle installation

Bundle installation

```
composer require league/oauth2-server-bundle
```

Minimal configuration

```yaml
# config/packages/league_oauth2_server.yaml
league_oauth2_server:
    authorization_server:
        private_key: '%kernel.project_dir%/var/oauth/private.key'
        encryption_key: 'def00000examplekey1234567890ab'

    resource_server:
        public_key: '%kernel.project_dir%/var/oauth/public.key'
```

Minimal configuration

```yaml
# config/packages/league_oauth2_server.yaml
league_oauth2_server:
    authorization_server:
        private_key: '%kernel.project_dir%/var/oauth/private.key'
        encryption_key: 'def00000examplekey1234567890ab'

    resource_server:
        public_key: '%kernel.project_dir%/var/oauth/public.key'
```

```yaml
# config/packages/security.yaml
security:
    firewalls:
        api_token:
            pattern: ^/token$
            security: false
        main:
            pattern: ^/api
            security: true
            stateless: true
            oauth2: true
```

Minimal configuration

```yaml
# config/packages/league_oauth2_server.yaml
league_oauth2_server:
    authorization_server:
        private_key: '%kernel.project_dir%/var/oauth/private.key'
        encryption_key: 'def00000examplekey1234567890ab'

    resource_server:
        public_key: '%kernel.project_dir%/var/oauth/public.key'
```

```yaml
# config/packages/security.yaml
security:
    firewalls:
        api_token:
            pattern: ^/token$
            security: false
        main:
            pattern: ^/api
            security: true
            stateless: true
            oauth2: true
```

```yaml
# config/routes.yaml
oauth2:
    resource: '@LeagueOAuth2ServerBundle/config/routes.php'
    type: php
```

Client entity

```php
use League\OAuth2\Server\Entities;

#[ORM\Entity]
class Client implements ClientEntityInterface
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 80)]
    private string $identifier;

    #[ORM\Column(length: 80)]
    private string $secret;

    public function getIdentifier(): string
    {
        return $this->identifier;
    }
}
```

Access token entity

```php
use League\OAuth2\Server\Entities;

#[ORM\Entity]
class AccessToken implements AccessTokenEntityInterface
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    private ?int $id = null;

    #[ORM\ManyToOne(targetEntity: Client::class)]
    private ClientEntityInterface $client;

    #[ORM\Column(type: 'json')]
    private array $scopes = [];

    public function getClient(): ClientEntityInterface
    {
        return $this->client;
    }
}
```

Protected API endpoint

```php
// src/Controller/ApiController.php
class ApiController extends AbstractController
{
    #[Route('/api/protected')]
    #[IsGranted('ROLE_OAUTH2_<scope>')]
    public function protectedEndpoint(): JsonResponse
    {
        return $this->json(['data' => 'Protected resource']);
    }
}
```

OAuth2 vs OIDC

Open**ID** **C**onnect standardizes **user authentication on top of OAuth2**.

It harmonizes the way to **retrieve user related resources** (i.e. profile, email, etc.)

OIDC

You can do **authentication using OAuth2**,
but user data are returned in a **non-standard** way.

OIDC standardizes this with the concept of **ID tokens**
that provide user information in a consistent format.

Authentication

OIDC extends OAuth2 with identity layer

OIDC extends OAuth2 with identity layer

🆔 **ID Tokens**: JWTs containing user identity claims

OIDC extends OAuth2 with identity layer

🆔 **ID Tokens**: JWTs containing user identity claims

📋 **Standard Claims**: sub, aud, iss, exp, etc.

OIDC extends OAuth2 with identity layer

🆔 **ID Tokens**: JWTs containing user identity claims

📋 **Standard Claims**: sub, aud, iss, exp, etc.

🔗 **Discovery**: Well-known configuration endpoint

OIDC extends OAuth2 with identity layer

🆔 **ID Tokens**: JWTs containing user identity claims

📋 **Standard Claims**: sub, aud, iss, exp, etc.

🔗 **Discovery**: Well-known configuration endpoint

🔐 **UserInfo**: Additional user data endpoint

OIDC Provider

Unlike OAuth2 server bundle that integrates an authorization server into the resource server, OIDC recommends to rely on a separated **OIDC Provider** that issues ID tokens, user info and access tokens.

JW* Explanation

Acronym	Full Name	Purpose	Metaphor	Is it a Token?
JWT	**J**SON **W**eb **T**oken	Defines the structure of the claims in the payload.	The letter or package contents	Yes, although it's never used without a JWS/JWE structure.
JWS	**J**SON **W**eb **S**ignature	Provides integrity and authenticity via a signature.	A clear envelope with a tamper-proof seal	Yes, a signed token.
JWE	**J**SON **W**eb **E**ncryption	Provides confidentiality via encryption.	A locked, opaque metal box	Yes, an encrypted token.

JW* Explanation

Acronym	Full Name	Purpose	Metaphor	Is it a Token?
JWA	**J**SON **W**eb **A**lgorithms	Defines the allowed cryptographic algorithms.	The list of approved lock/seal types	No, it's a list of names.
JWK(S)	**J**SON **W**eb **K**ey (**S**et)	A standard format for representing (set of) keys.	The actual key (set) and its metadata	No, it's a (set of) key(s) format.

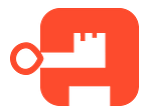
Unlike with OAuth2 server bundle,
we are not implementing our own OIDC Provider.

Open source OIDC Providers

Could also act as Identity Providers (IdP)

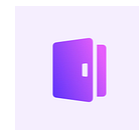

Authelia


Casdoor


Authentik


Hanko


Keycloak


LogTo


Ory Hydra


PocketID

Rauthy


SuperTokens


Zitadel

They all speak the same language: OIDC protocol

SaaS OpenID Connect Providers (OP)

Could also act as Identity Providers (IdP)


Auth0


AWS Cognito


Bare ID


Cidaas


Cloud-IAM


Microsoft Entra ID

(formerly Azure AD)


Firebase Auth


Hanko


LoginRadius


Okta


Quasr


SuperTokens


Zitadel

They all speak the same language: OIDC protocol

Need social SSO?

Most of listed OIDC providers support social SSO out of the box, by configuration of client credentials.

Symfony supports (most of) OIDC

Symfony supports (most of) OIDC

- 📚 Your API is still the **resource server**

Symfony supports (most of) OIDC

- 📚 Your API is still the **resource server**
- 🛂 Your OIDC Provider is the **authorization server**

Symfony supports (most of) OIDC

- 📚 Your API is still the **resource server**
- 🛂 Your OIDC Provider is the **authorization server**
- 👥 Users are **stored in the OIDC Provider**

Symfony supports (most of) OIDC

📚 Your API is still the **resource server**

🛂 Your OIDC Provider is the **authorization server**

👥 Users are **stored in the OIDC Provider**

🛰️ Your API **relies on the OIDC Provider** during the security process to **verify user identity and retrieve user information**

Symfony supports (most of) OIDC

🦸 Symfony supports 2 OIDC strategies:

Symfony supports (most of) OIDC

🦸 Symfony supports 2 OIDC strategies:

📶 Using `OidcUserInfoTokenHandler` to **validates tokens remotely** with the OIDC Provider

Symfony supports (most of) OIDC

🦸 Symfony supports 2 OIDC strategies:

- 📶 Using `OidcUserInfoTokenHandler` to **validates tokens remotely** with the OIDC Provider
- 📵 Using `OidcTokenHandler` to **validates tokens locally** with the OIDC Provider's public key

Symfony supports (most of) OIDC

🦸 Symfony supports 2 OIDC strategies:

📶 Using `OidcUserInfoTokenHandler` to **validates tokens remotely** with the OIDC Provider

📵 Using `OidcTokenHandler` to **validates tokens locally** with the OIDC Provider's public key

Both require HttpClient and Cache components.

```
composer require symfony/http-client symfony/cache
```

OidcUserInfoTokenHandler

```yaml
# config/packages/security.yaml
security:
    firewalls:
        main:
            # ...
            access_token:
                token_handler:
                    oidc_user_info: https://your-oidc-provider/realms/demo/protocol/openid-connect/userinfo
```

Symfony calls the user info endpoint to validate the token and retrieve user info during authentication.

With OIDC Discovery

```yaml
# config/packages/security.yaml
security:
    firewalls:
        main:
            # ...
            access_token:
                token_handler:
                    oidc_user_info:
                        base_uri: https://your-oidc-provider/realms/demo/
                        discovery:
                            cache:
                                id: cache.app
```

Symfony automatically discovers the OIDC endpoints (incl. user info) and cache it for better performance.

OidcTokenHandler

Require a library to manipulate JWT tokens:

```
composer require web-token/jwt-library
```

```yaml
# config/packages/security.yaml
security:
    firewalls:
        main:
            # ...
            access_token:
                token_handler:
                    oidc:
                        algorithms: ['ES256', 'RS256'] # Algorithms used to sign the JWS
                        audience: 'api-example' # Audience (`aud` claim): required for validation purpose
                        issuers: ['https://oidc.example.com'] # Issuers (`iss` claim): required for validation purpose
                        keyset: '{"keys":[{"kty":"...","k":"..."}]}' # A JSON-encoded JWK
```

Symfony validates the token locally with the provided JWKs, without relying on the OIDC Provider.

With OIDC Discovery

```yaml
# config/packages/security.yaml
security:
    firewalls:
        main:
            # ...
            access_token:
                token_handler:
                    oidc:
                        algorithms: ['ES256', 'RS256']
                        audience: 'api-example'
                        issuers: ['https://oidc.example.com']
                        discovery:
                            base_uri: https://www.example.com/realms/demo/
                            cache:
                                id: cache.app
```

Symfony automatically discovers the OIDC JWKs and cache them for better performance.

Token introspection?

Rate Limiting

Symfony Rate Limiter Overview

Symfony Rate Limiter Overview

- Built-in rate limiting component in Symfony

Symfony Rate Limiter Overview

- 🛡️ Built-in rate limiting component in Symfony
- 🎯 Protects against brute force and DoS attacks

Symfony Rate Limiter Overview

🛡️ Built-in rate limiting component in Symfony

🎯 Protects against brute force and DoS attacks

🔧 Flexible configuration for different use cases

Symfony Rate Limiter Overview

- 🛡️ Built-in rate limiting component in Symfony
- 🎯 Protects against brute force and DoS attacks
- 🔧 Flexible configuration for different use cases
- 📦 Available since Symfony 5.3

Symfony Rate Limiter Overview

- 🛡️ Built-in rate limiting component in Symfony
- 🎯 Protects against brute force and DoS attacks
- 🔧 Flexible configuration for different use cases
- 📦 Available since Symfony 5.3

> The Symfony Rate Limiter component provides a token bucket algorithm implementation for rate limiting requests.

Configuring Rate Limiters

Configuring Rate Limiters

📝 Define limiters in `config/packages/rate_limiter.yaml`

Configuring Rate Limiters

📝 Define limiters in `config/packages/rate_limiter.yaml`

🔢 Configure limits, intervals, and storage

Configuring Rate Limiters

- 📝 Define limiters in `config/packages/rate_limiter.yaml`
- 🔢 Configure limits, intervals, and storage
- 🎯 Different limiters for different endpoints

Configuring Rate Limiters

- 📝 Define limiters in `config/packages/rate_limiter.yaml`
- 🔢 Configure limits, intervals, and storage
- 🎯 Different limiters for different endpoints
- 🔄 Supports sliding window and fixed window algorithms

Configuring Rate Limiters

📝 Define limiters in `config/packages/rate_limiter.yaml`

🔢 Configure limits, intervals, and storage

🎯 Different limiters for different endpoints

🔄 Supports sliding window and fixed window algorithms

```yaml
# config/packages/rate_limiter.yaml
framework:
    rate_limiter:
        api:
            policy: 'token_bucket'
            limit: 100
            rate: { interval: '1 second', amount: 10 }
```

Using Rate Limiters in Controllers

Using Rate Limiters in Controllers

- 🎯 Apply to specific controller actions

Using Rate Limiters in Controllers

- 🎯 Apply to specific controller actions
- 🛡️ Automatic HTTP 429 response when limit exceeded

Using Rate Limiters in Controllers

- 🎯 Apply to specific controller actions
- 🛡️ Automatic HTTP 429 response when limit exceeded
- 🔧 Customize response behavior

Using Rate Limiters in Controllers

- 🎯 Apply to specific controller actions
- 🛡️ Automatic HTTP 429 response when limit exceeded
- 🔧 Customize response behavior
- 📊 Access limiter state and headers

Using Rate Limiters in Controllers

```php
class ApiController
{
    public function __construct(
        private RateLimiterFactory $apiLimiter,
    ) {}

    public function endpoint(Request $request): JsonResponse
    {
        if (!$this->apiLimiter->create($request->getClientIp())->consume()->isAccepted()) {
            throw new TooManyRequestsHttpException(); // with Retry-After header
        }

        // ... handle endpoint logic
        return new JsonResponse($data);;
    }
}
```

 API Gateway, the surprise!

Are last topics your API responsibility?

Authentication, authorization, rate limiting and globally securing and protecting your APIs is a full-time job!

API Gateways or API Management platforms are here to help you with that!

Pick your favorite!

Open-source or commercial, self-hosted or SaaS, there is a large choice!

Apisix

Gravitee

Kong

KrakenD

Otoroshi

Tyk

Zuplo

Delegate!

Delegate!

- 🔐 Authentication / authorization

Delegate!

- 🔐 Authentication / authorization
- ⏱️ Rate limiting

Delegate!

- 🔐 Authentication / authorization
- ⏱️ Rate limiting
- 🪵 Logging

Delegate!

- 🔐 Authentication / authorization
- ⏱️ Rate limiting
- 🪵 Logging
- 📊 Monitoring

Delegate!

🔐 Authentication / authorization

⏱️ Rate limiting

🪵 Logging

📊 Monitoring

… and more!

Delegate!

🔐 Authentication / authorization

⏱️ Rate limiting

🪵 Logging

📊 Monitoring

… and more!

Most of API Gateways offers those basic feature out of the box and finely configurable.

Key Takeaways

Key Takeaways

1. Standards Exist — Use Them

JWT, OAuth 2.0, OIDC are battle-tested.
Don't invent your own auth scheme.

Key Takeaways

1. Standards Exist – Use Them

JWT, OAuth 2.0, OIDC are battle-tested.
Don't invent your own auth scheme.

2. Modern Tools Make It Easy

Symfony Security, League Oauth2 Server, OIDC Providers.
Most of the hard work is done for you.

Key Takeaways

1. Standards Exist — Use Them

JWT, OAuth 2.0, OIDC are battle-tested.
Don't invent your own auth scheme.

2. Modern Tools Make It Easy

Symfony Security, League Oauth2 Server, OIDC Providers.
Most of the hard work is done for you.

3. Security Is a Feature

Not an afterthought. Not a "nice to have."
Build it in from day one.

Key Takeaways

1. Standards Exist – Use Them

JWT, OAuth 2.0, OIDC are battle-tested.
Don't invent your own auth scheme.

2. Modern Tools Make It Easy

Symfony Security, League Oauth2 Server, OIDC Providers.
Most of the hard work is done for you.

3. Security Is a Feature

Not an afterthought. Not a "nice to have."
Build it in from day one.

4. Equip your APIs

API Gateway, API Management platform.
Do not try to scale what could be delegated

Key Takeaways

1. Standards Exist – Use Them

JWT, OAuth 2.0, OIDC are battle-tested.
Don't invent your own auth scheme.

2. Modern Tools Make It Easy

Symfony Security, League Oauth2 Server, OIDC Providers.
Most of the hard work is done for you.

3. Security Is a Feature

Not an afterthought. Not a "nice to have."
Build it in from day one.

4. Equip your APIs

API Gateway, API Management platform.
Do not try to scale what could be delegated

Resources

Specifications

RFC 6749 – OAuth 2.0

RFC 7519 – JWT

OpenID Connect – OIDC Spec

RS256 vs HS256

Documentation

Symfony Security

OAuth2 Server Bundle

Tools

jwt.io – JWT Debugger

OAuth 2.0 Playground

🙏 Thank you!

Questions?

Feedback

ConFoo